

Admissions Workflow Automation

Requirements & Solution Design Document

Take-home exercise — Senior Frontend Engineer (Full-Stack)

Prepared for: Toddle

Prepared by: Ankit Meena

Date: 22 August 2026

Reference: workflow-14.netlify.app (Admissions workflows — example task brief)

Table of Contents

Executive Summary.....	3
Part A — Requirements.....	4
A.1 Problem Statement.....	4
A.2 Goals.....	4
A.3 Users & Personas.....	4
A.4 Functional Requirements.....	5
A.5 Non-Functional Requirements.....	6
A.6 Assumptions & Out of Scope.....	7
Part B — Admission Workflows.....	8
B.1 Workflow 1 — Admission Fee Payment Reminder.....	8
B.2 Workflow 2 — Post-Offer → Enrolment.....	11
B.3 Workflow 3 — Application Review & Staff Assignment.....	14
B.4 Workflow 4 — Scholarship Application Review.....	17
B.5 Workflow 5 — Applicant Interview Scheduling.....	20
Part C — Solution Design.....	21
C.1 Design Principles.....	21
C.2 System Architecture.....	21
C.3 Core Domain Model.....	22
C.4 Step Type Specification.....	22
C.5 Execution Engine (Backend Overview).....	24
C.6 Frontend Architecture.....	25
C.7 Backend Architecture & Tech Stack.....	30
C.8 Security, Privacy & Compliance.....	31
C.9 Additional Step Types & Features Worth Adding.....	31
C.10 Publish & Rollout.....	32
Appendix 1 — Step Type Legend.....	33

Executive Summary

This document sets out ten step types for the builder — the seven already available (Trigger, Delay, Branches, Communicate, Set status, Go to action, Goal) plus three additions — Request Document, Assign to Staff / Create Task, and Schedule Interview — that close the biggest gaps in the current step set. Five workflows are built from this palette: the three examples in the task brief, plus two that apply the same step types to different situations — reviewing scholarship applications and scheduling applicant interviews.

Part A sets out the requirements. Part B presents the five workflows: Workflows 1 and 4 in full (diagram plus a step-by-step table of why each step belongs), Workflows 2 and 3 summarised beside their diagram, and Workflow 5 in a short paragraph. Part C covers the solution design, with the frontend architecture (C.6) as the main focus — the step type registry, the composition-based form and field design system (C.6.10), how a workflow's JSON drives the canvas, the draft/publish save flow, and how the canvas lays itself out — alongside enough of the data model, execution engine and backend stack (C.2, C.3, C.5, C.7) to place the frontend in its whole-system context.

Part A — Requirements

A.1 Problem Statement

Admissions is a time-bound, multi-party process: an applicant moves through stages — submitted, under review, offered, accepted, enrolled — and at each stage the school needs to nudge people (fee reminders, enrolment reminders), inform staff (assign a reviewer, escalate a stalled case), and keep the application's status in sync with what has actually happened. Today this kind of thing is typically done by hand or via one-off scripts that only engineering can change. The product's promise is that an admissions officer can configure flows like these visually, publish them, and trust that they keep running correctly for every application — including waits that span days — without a developer in the loop.

A.2 Goals

- Let admissions staff build, edit and publish workflows without writing code.
- Guarantee that a published workflow executes reliably for every application that matches its trigger, including multi-day waits, server restarts, and deploys.
- Keep each flow legible to a non-engineer reading it on the canvas — which is why the reasoning behind each step matters as much as the step itself.
- Provide enough building blocks to express retries, escalation, document collection and staff assignment using step types in the builder, rather than needing a developer to add a new step type for every new scenario.
- Let Communicate messages pull in structured data (applicant name, deadline, programme, assignee, etc.) as merge-field variables, scoped to who the message is actually being sent to.
- Surface structural mistakes — an unreachable step, a loop with no way out, a missing required field — before a flow is published, not after it has emailed a family.

Non-goals for this document

- Payment processing itself — a separate fee/payment system is assumed to exist and to expose a fee status the workflow can read.
- The admissions record system's full field schema.
- Pixel-level visual design of the canvas — the provided demo already establishes this.

A.3 Users & Personas

- Admissions Officer / Workflow Builder — builds and publishes flows on the canvas. Not a developer; needs confidence a published flow will simply keep working.
- Reviewing Staff / Faculty — receives internal notifications a workflow assigns to them; acts outside the tool, in their inbox or the student system.
- Applicant / Parent or Guardian — receives communications from the workflow; never opens the builder.

- School Admin (super-user) — manages who can build vs. only view, and looks at run history across workflows.

A.4 Functional Requirements

ID	Requirement	Rationale
FR-1	Visually build, edit and publish workflows on a canvas, using the step types below, with no code.	Core promise of the product for non-technical admissions staff.
FR-2	Support the seven step types: Trigger, Delay, Branches, Communicate, Set status, Go to action, Goal.	These are the atomic building blocks every workflow in Part B is composed from.
FR-3	A Trigger fires once per matching event per application, and each firing starts an independent run.	Hundreds of applicants can be mid-flow at once without their runs interfering with each other.
FR-4	Delay steps resume automatically after the wait, even across days and server restarts.	Fee and enrolment reminders depend on multi-day waits; a missed resume is a missed reminder.
FR-5	Branches can test the application's current status, a date/time comparison, a run-scoped counter, a document's upload status, and a task's completion status.	Every workflow in Part B needs at least one of these to decide "remind again" vs. "escalate".
FR-6	Go to action can jump to another step in the same run, with a built-in cap on repeat count and a target the author picks by its visible step number.	Reminder loops must repeat a bounded number of times, then fall through to escalation.
FR-7	Communicate supports merge-field variables (e.g. {{applicant_name}}, {{deadline_date}}) and can address the applicant, guardian, both, or internal staff — the variables on offer adapt to who is selected.	The same step type drives both applicant-facing reminders and internal staff notifications, without risking a staff-only field leaking into a family-facing email.
FR-8	Set status writes back to the application's status field, readable by the rest of the school's system.	Staff dashboards and other workflows depend on status being a single source of truth.
FR-9	Goal ends a run and records a label for why it ended (e.g. "Fee collected" vs. "Escalated").	Without a labelled end state, staff can't tell a healthy completion from a stalled one in run history.
FR-10	Every run keeps a visible history of which steps executed, when, and with what result, per application.	Staff need to answer "what happened to this applicant's reminders" without reading raw logs.
FR-11	Workflows distinguish draft vs. published; edits don't affect runs already in progress until published, and a published workflow can be unpublished to stop it starting new runs.	Staff will iterate on wording and timing; in-flight applicants shouldn't be moved onto a half-edited flow.
FR-12	Role-based permissions distinguish who can edit/publish a workflow from who can only view its run	Non-technical staff should see status without being able to break a live, applicant-facing flow.

ID	Requirement	Rationale
	history.	
FR-13	A Request Document step lets a workflow ask an applicant/guardian to upload one or more files against a checklist, and exposes an upload status Branches can read.	Today "documents submitted?" has to be inferred from a field some other system sets; a first-class step makes Workflow 2 fully self-serve (see C.4).
FR-14	An Assign to Staff / Create Task step assigns an application to a person and creates a tracked task with its own due date and completion state, which Branches can read.	Lets "reviewed?" be answered by task completion rather than elapsed time alone, and makes reassignment/escalation trackable (see C.4, Workflow 3).
FR-15	Create, rename, and delete workflows from the builder (deleting one also removes its runs, tasks and history), and move a workflow between draft and published.	Staff manage their own library of flows without engineering or direct database access.
FR-16	Undo and redo canvas edits, from a toolbar button or the keyboard, scoped to the workflow being edited.	Building a flow is exploratory; a mis-drag or an accidental delete should be one step to reverse.
FR-17	Check a workflow's structure and surface any problems — an unreachable step, an unconnected branch, a loop with no exit, a step with required fields still empty — and block publishing while errors remain.	A mistake here is public-facing: it emails real families. It should be caught in the builder, not in production.

A.5 Non-Functional Requirements

ID	Requirement	Why it matters
NFR-1	Durability — a scheduled Delay survives a server restart or deploy.	Multi-day waits are central to every workflow here; losing state mid-wait silently breaks a live reminder.
NFR-2	Idempotency — retrying a failed step never sends a duplicate email or double-applies a status change.	A duplicate "your fee is overdue" email erodes trust in the school's communication.
NFR-3	Scale — thousands of applications can be mid-flow concurrently, each independently scheduled.	A single admissions cycle can involve thousands of applicants triggering the same workflow.
NFR-4	Auditability — every step execution, email sent, and status change is logged with a timestamp.	Needed to answer support questions ("why didn't this family get a reminder") and for compliance.
NFR-5	Privacy — student/guardian PII is handled per the applicable education-data regulation.	Both the vendor and the school carry compliance obligations for this student data.
NFR-6	Usability — a non-engineer can read a published flow and understand it without documentation.	This is the entire value proposition of a visual builder over a script.

ID	Requirement	Why it matters
NFR-7	new step types can be added without redesigning existing flows, and in the common case without touching frontend code at all	Section C.9 proposes several; today's seven step types should not be assumed final.

A.6 Assumptions & Out of Scope

- The admissions system already models an "Application" with a status field the workflow engine can read and write.
- Fee collection lives in a separate payment system; the workflow only reads/writes a fee status, it does not process payments itself.
- Email is the only communication channel in the builder today; SMS/WhatsApp are proposed as future work (C.9), not assumed here.
- This document does not re-specify the canvas's visual design, which the provided demo already establishes.

Part B — Admission Workflows

Workflows 1, 2 and 3 match the three examples in the task brief; Workflows 4 and 5 are additional flows — reviewing scholarship applications, and scheduling applicant interviews — the latter exercising the tenth step type, Schedule Interview. Workflows 1 and 4 are shown in full (diagram and step-by-step table); Workflows 2 and 3 keep their diagram with a short written summary; Workflow 5 is summarised in a paragraph. Dashed arrows in the diagrams are loop-backs (Go to action).

B.1 Workflow 1 — Admission Fee Payment Reminder

Reminds an accepted applicant's family to pay the admission fee, escalating to a human after a bounded number of automated reminders instead of nagging indefinitely.

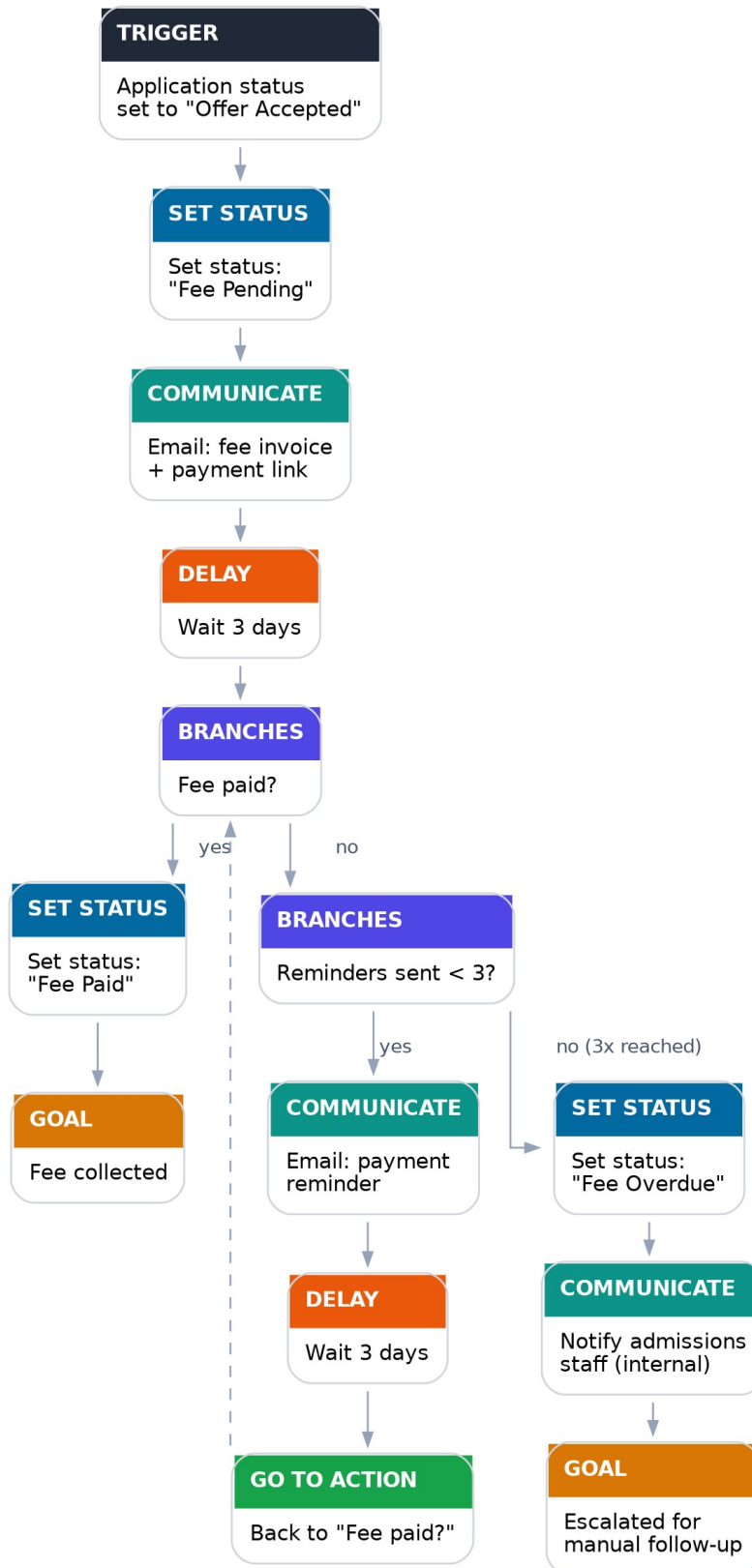


Figure B.1 — Admission fee payment reminder

#	Type	Action	Why it belongs here
1	Trigger	Fires when an application's status becomes "Offer Accepted".	This is the earliest point a fee is actually owed — triggering any earlier would ask for money before there's an offer to pay for.
2	Set status	Set status to "Fee Pending".	Gives staff and other workflows one source of truth that a fee is now outstanding, without waiting for the first email to go out.
3	Communicate	Email the fee invoice and payment link to the applicant and guardian, using {{applicant_name}}, {{fee_amount}} and {{due_date}}.	The actual ask — must happen before any wait, so the Delay below measures "time since we asked", not something else.
4	Delay	Wait 3 days.	Gives the family a reasonable window to pay before being chased.
5	Branches	"Fee paid?"	Re-checks reality before doing anything else — if it's already paid, the flow should stop, not blindly send the next reminder.
6	Set status (yes)	Set status to "Fee Paid".	Closes the loop on status so the rest of the system knows, without anyone re-reading emails.
7	Goal (yes)	"Fee collected".	A clearly labelled, successful completion in the run history.
8	Branches (no)	"Reminders sent < 3?"	Bounds the loop — without this the flow nags indefinitely and never escalates.
9	Communicate (yes)	Send a reminder email.	The actual nudge.
10	Delay	Wait 3 days.	Same reasoning as step 4, repeated per reminder.
11	Go to action	Jump back to "Fee paid?".	Re-checks payment status before deciding to send yet another reminder or stop.
12	Set status (no, 3x reached)	Set status to "Fee Overdue".	Signals to staff — and any downstream workflow, e.g. an enrolment hold — that this case is now stuck.
13	Communicate	Notify admissions staff internally, using {{applicant_name}} and {{application_id}}.	Automated reminders have run their course; a human should decide the next step (e.g. call the family).
14	Goal	"Escalated for manual follow-up".	A distinct end state from "Fee collected", so staff scanning run history can immediately see who needs a human touch.

B.2 Workflow 2 — Post-Offer → Enrolment

Carries an applicant from receiving an offer through accepting it and completing enrolment, with a bounded reminder loop at the accept-offer stage and a first-class Request Document step (see C.4) at the document-collection stage.

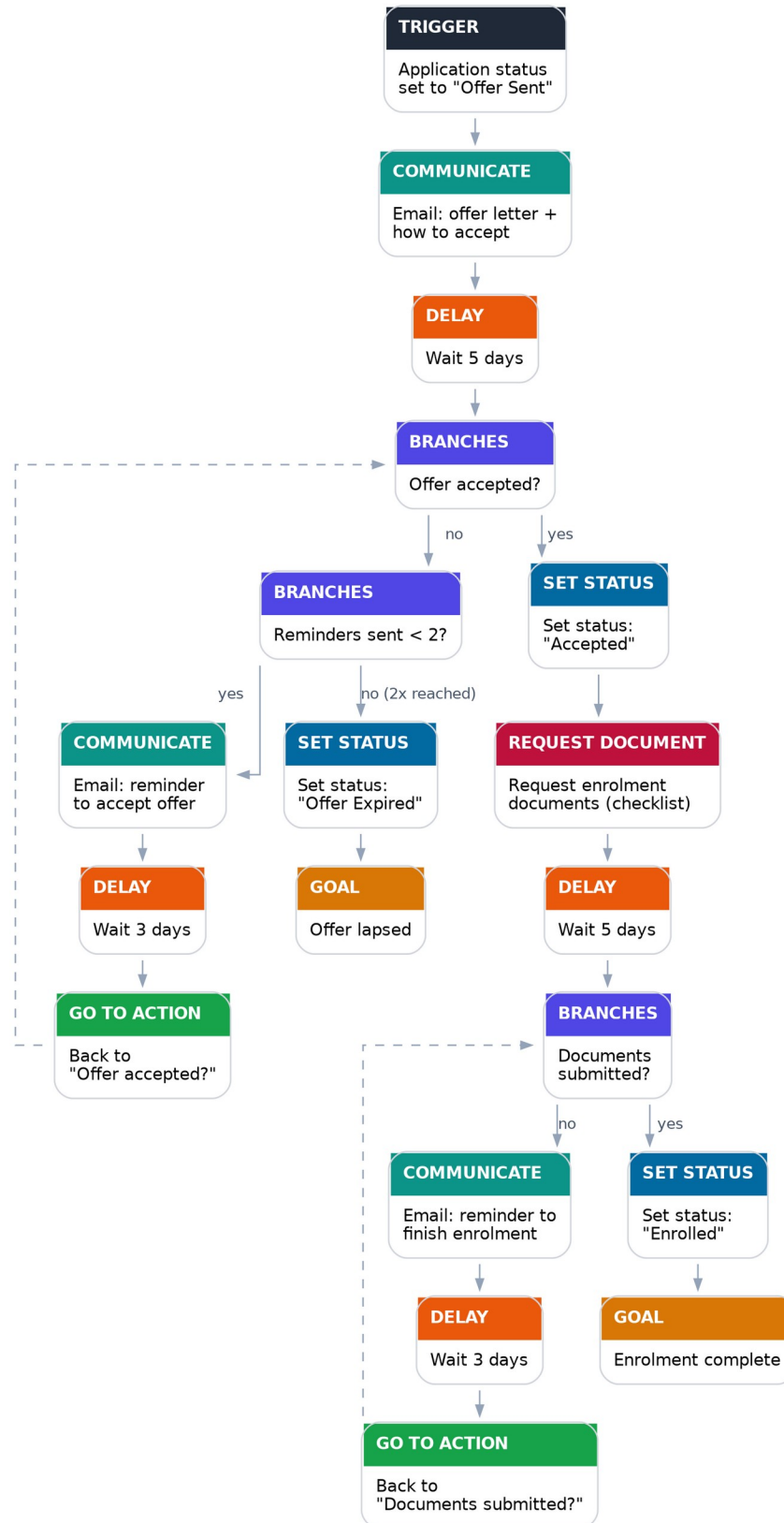


Figure B.2 — Post-offer to enrolment

In full, the diagram above reuses Workflow 1's bounded reminder-and-escalate pattern at the accept-offer stage, then — once the offer is accepted — collects enrolment paperwork with a Request Document step and a second bounded reminder loop before reaching “Enrolment complete”.

B.3 Workflow 3 — Application Review & Staff Assignment

Attaches every new application to a reviewer via a tracked task and keeps it on track against an internal SLA, escalating once — by reassigning the task — if the review is late rather than letting an application sit unreviewed indefinitely.

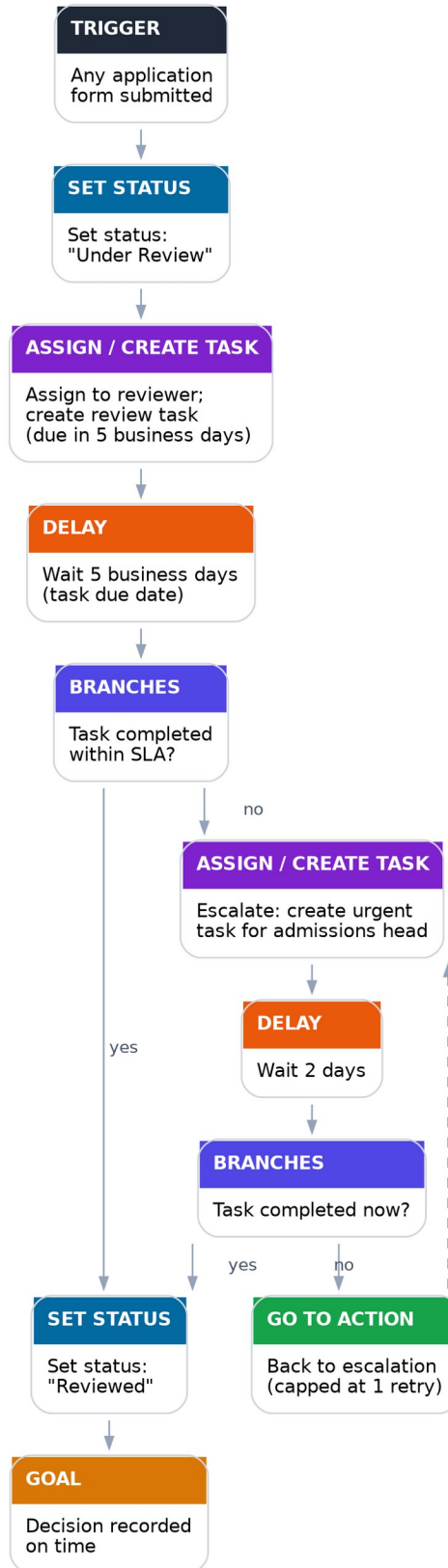


Figure B.3 — Application review & staff assignment

In full, the diagram above attaches the application to a reviewer through a tracked task with an SLA, reads that task's completion status with a Branches step, and escalates once — reassigning the task to the admissions head — if the review runs late.

B.4 Workflow 4 — Scholarship Application Review

A fourth flow, beyond the three in the brief, that shows the same step types applied to a different situation — and the one flow that uses all nine step types together. Its trigger is scoped to the scholarship form, so an ordinary application or enquiry submission doesn't start it. It chases missing financial-aid documents with a bounded reminder loop, then routes the application to the scholarship committee for a decision, escalating once to the financial aid director if the review runs late. Both halves reuse patterns already established in Workflows 2 and 3.

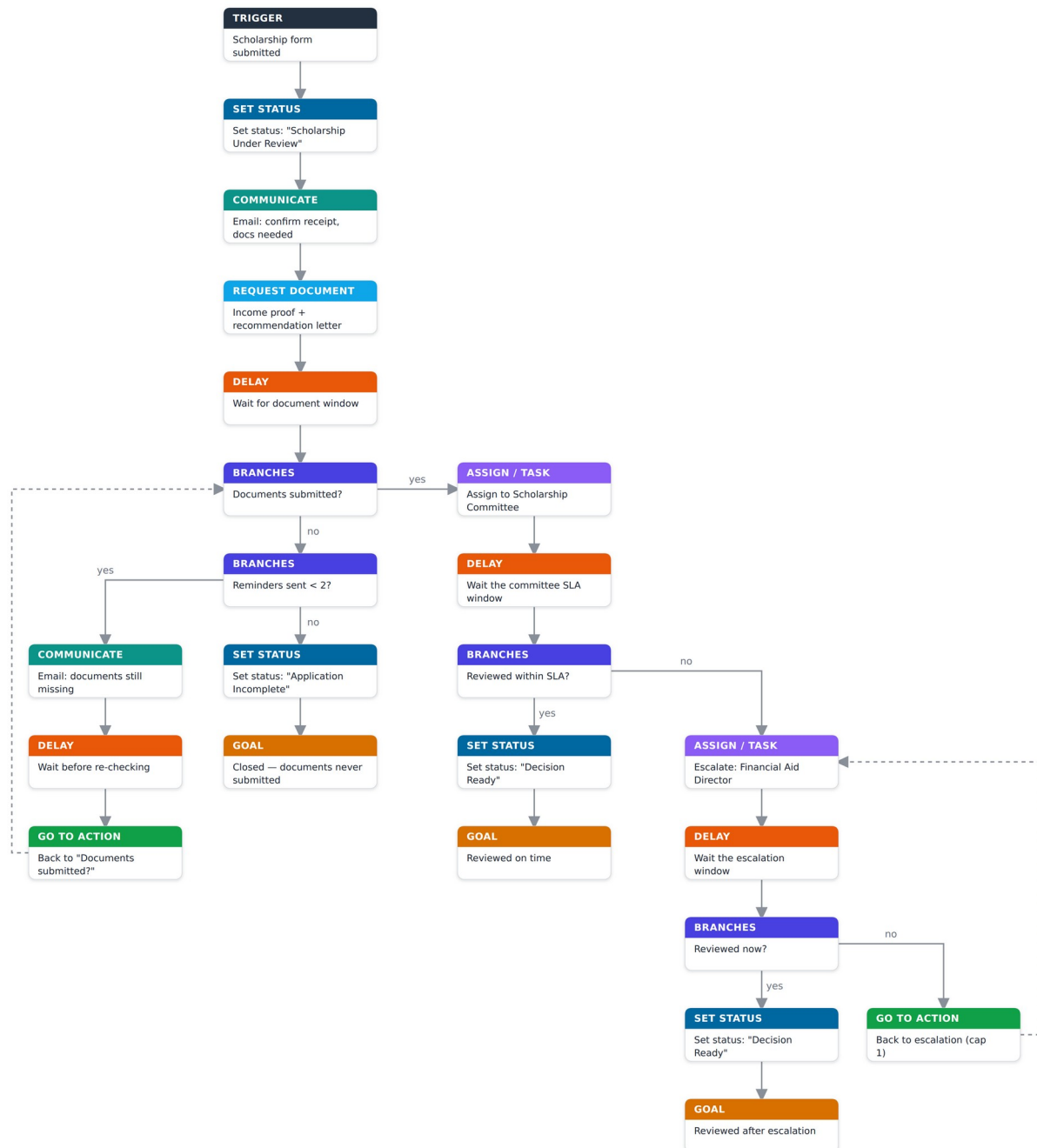


Figure B.4 — Scholarship application review

#	Type	Action	Why it belongs here
1	Trigger	Fires when the scholarship form is submitted.	Scoped to the scholarship form, so only scholarship applications start this flow.
2	Set status	Set status to "Scholarship Under Review".	Signals straight away that the application is active in the scholarship pipeline.
3	Communicate	Confirm receipt to applicant and guardian, noting that documents are still needed.	Sets expectations before the document request arrives.
4	Request Document	Ask the guardian for income proof and a recommendation letter, due in 5 days.	A tracked request the branch below can read directly, instead of inferring it from another field.
5	Delay	Wait out the document window.	Gives the family time to gather and submit before anything is checked.
6	Branches	"Documents submitted?" — reads the request's upload status.	If the documents are in, skip straight to committee review.
7	Branches (no)	"Reminders sent < 2?"	Bounds the document-chase loop so it doesn't nag indefinitely.
8	Communicate (yes)	Remind the guardian which documents are still outstanding.	The nudge itself.
9	Delay	Wait before re-checking.	Gives the reminder time to land.
10	Go to action	Jump back to "Documents submitted?" (cap 5).	Re-checks before sending another reminder or giving up.
11	Set status (no)	Set status to "Scholarship Application Incomplete".	Records that the required paperwork never arrived.
12	Goal	"Closed — required documents never submitted".	A clearly labelled non-completion.
13	Assign / Create Task (yes)	Assign the application to the Scholarship Committee, due in 5 days.	A committee decision needs an owner; a tracked task is reassignable and can be checked against an SLA.
14	Delay	Wait out the committee's review window.	Mirrors the task's due date.
15	Branches	"Reviewed within SLA?" — reads	Checks whether the review actually happened rather than

#	Type	Action	Why it belongs here
		the task's completion status.	assuming.
16	Set status (yes)	Set status to "Scholarship Decision Ready".	The review is done and on time.
17	Goal (yes)	"Reviewed on time".	A clean, on-time completion.
18	Assign / Create Task (no)	Escalate with an urgent task for the Financial Aid Director, due in 2 days.	The committee missed its window; escalate through the same trackable mechanism.
19	Delay	Wait the shorter escalation window.	A tighter grace period for an already-late case.
20	Branches	"Reviewed now?" — reads the escalated task's status.	Re-checks before escalating again or accepting the case as done.
21	Set status (yes)	Set status to "Scholarship Decision Ready".	One consistent status regardless of which path reached it.
22	Goal (yes)	"Reviewed after escalation".	Distinguishes an escalated completion from an on-time one.
23	Go to action (no)	Jump back to the escalation step, capped at one repeat.	Bounds the escalation loop with a hard limit.

B.5 Workflow 5 — Applicant Interview Scheduling

A fifth flow, beyond the four above, built to exercise the Schedule Interview step end-to-end. When an application is marked “Reviewed”, it books an interview, invites the family, then chases with a bounded reminder loop and escalates to admissions staff if the interview is never completed — reusing the reminder-and-escalate shape already established in Workflows 1 and 3.

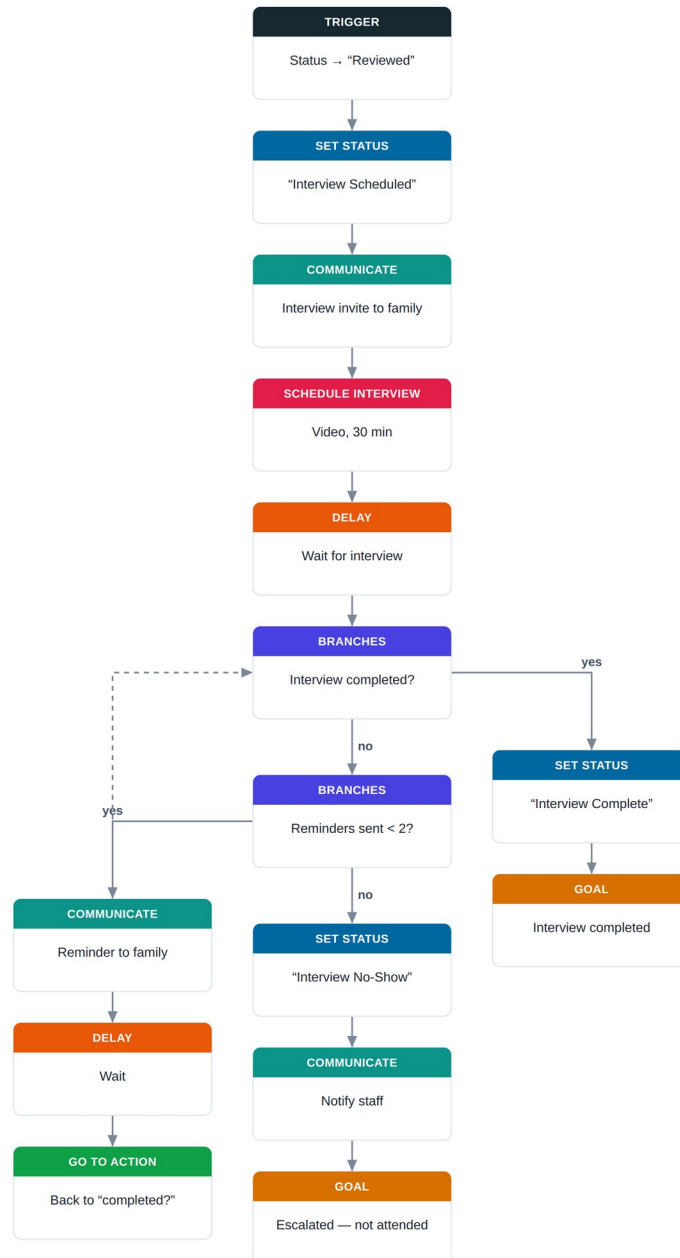


Figure B.5 — Applicant interview scheduling

Part C — Solution Design

C.1 Design Principles

- Configuration over code — every workflow in Part B is expressed using the builder's step types: the original seven, plus the two proposed additions (Request Document, Assign to Staff / Create Task) specified in C.4. No step in Part B requires custom engineering to wire up.
- Read reality, don't trust memory — every Branches condition re-reads live state at the moment it's reached (application status, a document request's upload status, a task's completion status), rather than assuming nothing changed during a Delay.
- Bounded loops by default — every Go to action in Part B pairs with a Branches counter that eventually breaks out to escalation; the platform should enforce a hard ceiling as a safety net regardless of what the flow author configures (see C.4).
- One workflow, many independent runs — the same published workflow definition drives potentially thousands of concurrent, independent per-application runs (see C.3).
- The canvas is a projection of the workflow, not a second copy of it — a workflow is steps[] plus edges, and the canvas computes each node's position from that graph on every render (C.6.3), so there are no coordinates to keep in sync.
- Validate before publishing — a workflow's structure is checked as it is edited and again when it is published (C.6.8), so a broken flow can't reach a real family.

C.2 System Architecture

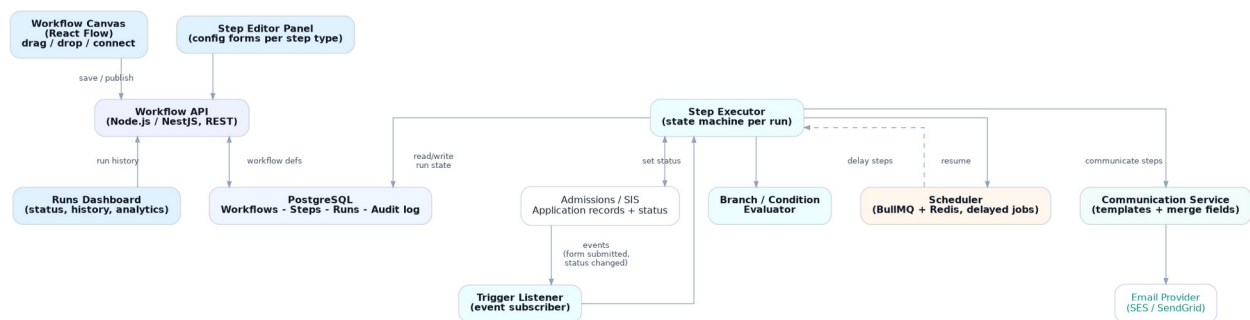


Figure C.1 — System architecture overview

The canvas (frontend) only ever talks to the Workflow API — it saves definitions and reads run history, but never executes anything itself. The Trigger Listener turns admissions-system events into new runs; the Step Executor is a small state machine that walks a run through its steps, handing off to the Scheduler whenever it hits a Delay so the wait survives restarts (NFR-1), and to the Communication Service whenever it hits a Communicate step. Set status writes flow back into the admissions/SIS system, which is also where Branches conditions are read from at execution time.

C.3 Core Domain Model

Entity	Key fields	Purpose
Workflow	id, name, trigger_config, status (draft/published), draft_revision, steps[]	The authored definition an admissions officer edits on the canvas. Only ever draft or published — no version history to manage.
Step	id, type, config (JSON), next[] (port → target-step references)	One node in the workflow graph. Config shape depends on type. A step carries no canvas position — the builder computes each node's position from the graph as it renders (C.6.3).
WorkflowRun	id, workflow_id, application_id, current_step_id, status (running/waiting/completed/escalated), started_at, steps_snapshot	One per application per trigger firing. Carries its own copy of the steps it needs (steps_snapshot) so a later edit to the workflow can't affect a run already in progress — without needing a version history.
RunStepHistory	id, run_id, step_id, executed_at, result, detail	The audit trail. Answers "what happened to this applicant's flow" (FR-10, NFR-4).
DocumentRequest	id, run_id, step_id, requested_files[] (checklist), uploaded_files[], status (pending/submitted), requested_at, submitted_at	Backs the Request Document step (FR-13). Branches reads status to decide "documents submitted?" instead of inferring it from an external field.
Task	id, run_id, step_id, assignee, title, due_at, completed_at, status (open/completed)	Backs the Assign to Staff / Create Task step (FR-14). Branches reads status/completed_at to decide "task completed?".
Application (external)	id, status, fee_status, documents[], assigned_reviewer, ...	The existing admissions record the workflow reads conditions from and writes Set status back to.
Interview	id, run_id, step_id, application_id, mode, room, duration_mins, status (scheduled/completed/no_show), scheduled_at, completed_at	Backs the Schedule Interview step. Branches reads status via the interview-status condition to decide "interview completed?", mirroring DocumentRequest and Task.

C.4 Step Type Specification

Step type	Purpose	Key config	Execution behaviour / edge cases
Trigger	Starts a new run.	Event type (e.g. form submitted, status changed to X); optional filter.	One run per matching event per application. Each run copies the steps it needs at that moment (steps_snapshot, C.3), so if the workflow is edited and republished later, runs already in progress keep executing the steps they started with (FR-11).

Step type	Purpose	Key config	Execution behaviour / edge cases
Delay	Pause a run.	Fixed duration (e.g. "3 days") or computed ("until date field").	Implemented as a durable scheduled job, not an in-memory timer, so it survives restarts (NFR-1). Resuming must be idempotent — a job firing twice must not re-execute the step twice (NFR-2).
Branches	Split execution on a condition.	Condition type: status equals, field comparison, document/task status, or a run-scoped counter (e.g. "times this step has executed"); true/false next step.	Evaluated against live state at the moment the branch is reached — not at trigger time — or a fee paid (or document uploaded) mid-wait would be missed.
Communicate	Send an email.	Recipient (applicant / guardian / applicant & guardian / assigned staff), message body, merge-field variables scoped to the chosen recipient (see below).	Must be idempotent per (run, step, attempt) — a retried send must not double-email a family. A template render failure should surface, not silently drop the step. The variable picker only offers fields valid for the selected recipient.
Set status	Change the application's status.	Target field, new value.	Should be the one place status changes so two workflows can't race and leave status inconsistent — likely needs an optimistic-lock or defined "last writer wins" rule.
Go to action	Jump to another step in the same run.	Target step, chosen from a list that shows each step by its visible number and title, plus a maximum-iteration count.	Without a hard cap this can loop forever. The builder should warn if a cycle has no reachable Branches condition able to break out of it.
Goal	Marks a run finished.	Free-text label (e.g. "Fee collected", "Escalated").	Should support more than one distinct Goal per workflow, as in every flow in Part B, so run history can distinguish a clean finish from an escalation at a glance.
Request Document	Ask for one or more files and track their upload.	Recipient (applicant/guardian), checklist of requested files, optional due date.	Creates a DocumentRequest record the moment the step runs (C.3). Branches reads its status ("pending"/"submitted") at the moment it's reached. A reminder for an outstanding request is still an ordinary Communicate + Delay + Branches loop (see Workflow 2), not reinvented here.
Assign to Staff / Create Task	Assign the application to a person and create a tracked task.	Assignee (a person or a role, e.g. "Admissions Head"), task title, due date (fixed or relative, e.g. "+5 business days").	Creates a Task record (C.3). Branches reads its completion status/timestamp rather than inferring completion from elapsed time alone. Escalation is just another instance of this step, targeting a different assignee (see Workflow 3).

Step type	Purpose	Key config	Execution behaviour / edge cases
Schedule Interview	Book an interview for the applicant and track it.	Mode (video / in person), room (in-person only), duration in minutes.	Creates an Interview record the moment the step runs (C.3), like Request Document and Assign Task. A Branches interview-status condition reads whether it is scheduled, completed, or a no-show; a demo action marks an interview complete.

Communicate — recipient-scoped variables

The Communicate editor's merge-field picker is populated from a per-recipient-type schema, so it only ever offers variables that make sense for whoever the message is addressed to — a staff-only field such as `{{application_id}}` simply isn't an option when "Applicant" is selected, and vice versa.

Send to	Available variables
Applicant	<code>{{applicant_name}}</code> , <code>{{programme}}</code> , <code>{{deadline_date}}</code> , <code>{{portal_link}}</code>
Parent / Guardian	<code>{{guardian_name}}</code> , <code>{{applicant_name}}</code> , <code>{{programme}}</code> , <code>{{deadline_date}}</code> , <code>{{portal_link}}</code>
Applicant and guardian	Union of the Applicant and Parent/Guardian rows above
Assigned staff member	<code>{{staff_name}}</code> , <code>{{applicant_name}}</code> , <code>{{application_id}}</code> , <code>{{application_status}}</code> , <code>{{task_due_date}}</code>

C.5 Execution Engine (Backend Overview)

Kept brief since this document's focus is the frontend (C.6) — enough backend context to place it in the whole system, not a full engine spec.

- Each WorkflowRun is a small state machine: a Trigger Listener starts one per matching event, and a Step Executor walks it through its steps — continuing straight through Set status / Communicate / Branches, or handing off to a durable job queue (the Scheduler) whenever it hits a Delay, so multi-day waits survive a restart (NFR-1).
- Every step execution is recorded in RunStepHistory before its side effect fires, so a retried step can never double-send an email, double-write a status, or create a duplicate document request or task (NFR-2). A per-run loop-iteration cap on Go to action is the safety net behind the reminder loops in Part B.
- That's the extent of backend mechanics the frontend needs to reason about — the rest (queue choice, retry policy) sits behind the same Step Executor that the frontend's step type registry (C.6.1) already describes the public shape of.

C.6 Frontend Architecture

React + TypeScript, with React Flow driving the canvas and an automatic layout arranging it.

C.6.1 Step Type Registry

its config schema (plain JSON Schema), default config, icon, a portable colour token, category, a short summary template

```
// backend/stepTypes/communicate.ts
const configSchema = {
  type: "object",
  properties: {
    recipient: { type: "string", enum: ["applicant", "guardian", "both", "staff"] },
    message: { type: "string" },
  },
  required: ["recipient", "message"],
};

export const communicateStepType: StepTypeDefinition = {
  type: "communicate",
  label: "Communicate",
  icon: "mail",
  color: "teal",
  category: "communication",
  configSchema, // plain JSON Schema
  defaultConfig: { recipient: "applicant", message: "" },
  ports: { outputs: [{ id: "next" }] },
  execute: async (config, ctx) => { /* server-only, never sent to client */ },
};
```

There is no conversion step. configSchema is plain JSON Schema, authored directly inside the step type module. The `/api/step-types` endpoint returns it exactly as received, so the object the frontend validates against is byte-for-byte the same one the backend validates against — nothing to keep in sync, and no second copy to version.

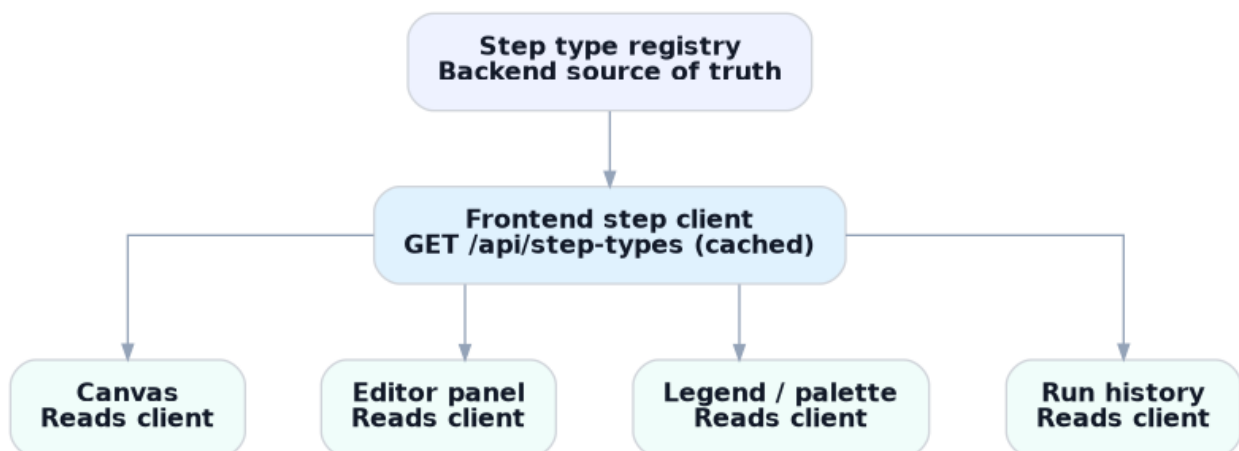


Figure C.2 — Step type registry, from backend source of truth to the four generic UI surfaces that consume it

The frontend fetches `/api/step-types` once (cached via React Query) and builds a local step client from it. Two small local maps translate the portable strings back into renderables — an icon-name map and a colour-token map — so the backend never needs to know about React components. Every consumer above (canvas, editor panel, legend/palette, run history) is written once, generically, against this client rather than against any specific step type; the Legend tab is literally a loop over it.

The palette's category grouping and ordering are served from the backend too (GET `/api/step-categories`) rather than hardcoded in the frontend, and a step type can carry a hidden flag to gate it out of the palette — so the catalog itself is data-driven, not just each step's config.

Most step types (Delay, Set status, Goal, Request Document, Assign to Staff) need only a handful of simple fields, so a generic SchemaForm component renders their editor directly from the fetched JSON Schema — validated client-side with ajv against the exact same JSON Schema object the backend re-validates with, so the two can never drift. Branches, Communicate, Trigger, Go to action and Schedule Interview need bespoke interaction and register a step editor in a small map keyed by step type; a step type with no entry falls back to the generic form automatically, so it is usable immediately rather than blocked on frontend work. The atoms and hooks that both the generic form and these bespoke editors are built from are described in C.6.10.

C.6.2 Extending the Pattern: Pluggable Sub-Behaviours

The same "declare once, register, let generic UI read it" pattern repeats one level deeper, inside the two step types that have pluggable sub-behaviour — this is what lets a developer customise an existing step type without touching its editor component:

- Branches' condition types (status equals, date comparison, run-scoped counter, document status, task status, interview status) are themselves a small registry. Adding a new condition later — e.g. "field changed in the last N minutes" — is an additive registration, not a change to the Branches editor.
- Communicate's recipient-to-variable mapping (the table in C.4) is a data-driven map keyed by recipient type, not a switch statement inside the editor. Adding a recipient (e.g. a secondary guardian) or a variable is a data change to that map, not a code change.

C.6.3 Canvas as a Config Projection

A workflow's steps[] — each step's config and its next-step references — is the only source of truth. A step does not store a canvas position: the workflow is a graph, and the canvas computes each node's coordinates from that graph as it renders, using a layered top-down layout. Positions stay consistent as steps are added, removed or reordered, and every edit changes the graph rather than a set of coordinates

```

function workflowConfigToGraph(workflow, stepTypes) {
  const nodes = workflow.steps.map(step => ({
    id: step.id,
    type: "stepNode",           // one React Flow node type, always
    position: step.position,
    data: { step, def: stepTypes[step.type] }, // the registry lookup
  }));
  const edges = workflow.steps.flatMap(step =>
    step.next.map(n => ({
      id: `${step.id}-${n.portId}-${n.targetId}`,
      source: step.id, sourceHandle: n.portId, target: n.targetId,
      style: step.type === "go_to_action" ? { strokeDasharray: "5 5" } : undefined,
    })))
  );
  return { nodes, edges };
}

function graphToWorkflowConfig(nodes, edges) {
  return nodes.map(n => ({
    id: n.id, type: n.data.step.type, config: n.data.step.config, position: n.position,
    next: edges.filter(e => e.source === n.id)
      .map(e => ({ targetId: e.target, portId: e.sourceHandle })),
  }));
}

```

React Flow only ever registers one node type — a generic `StepNode` — so it has no idea `Communicate` and `Delay` are different things; it reads `def.icon`, `def.color` and `def.summarize(step.config)` from the registry lookup to render each card. The number and labels of a node's output handles (Branches' yes/no, a linear step's single "next") come from the same `def.ports` metadata rather than a hardcoded check on step type.

Only one direction of that mapping is needed in practice. Edits are applied straight to `steps[]` through a small set of focused functions — insert a step, remove one, reorder one, retarget a Go to action, set the trigger, edit a step's config — so there is no canvas-to-config round-trip to keep correct, and each of those functions is easy to test on its own. Dragging a step is a reorder: drop it onto an edge to splice it into the chain (a marker shows where it will land), or drop it elsewhere and it settles back into its computed place.

C.6.4 Save Flow: Draft Autosave & Publish

Every canvas mutation — add a step, edit its config, drag it, rewire an edge — updates local state immediately for instant feedback, then passes through `graphToWorkflowConfig` and is pushed to the backend as a debounced autosave.

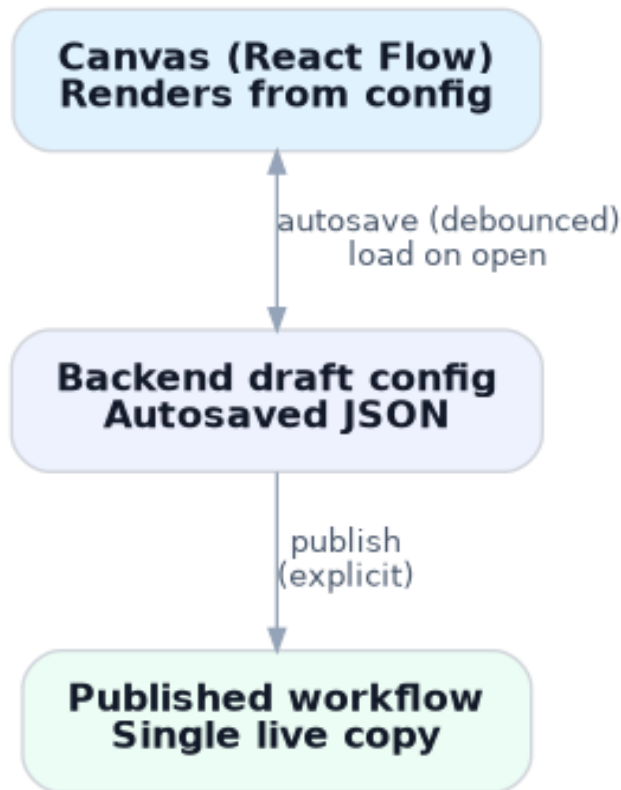


Figure C.3 — The canvas renders from the backend draft config and autosaves edits back to it; publishing is a separate, explicit action

- Autosave only ever touches the draft (PATCH /api/workflows/:id/draft), never the published copy — in line with the draft/publish distinction in FR-11 and C.10.
- if simultaneous multi-editor collaboration became a requirement.
- A lightweight revision number guards against stale writes — each save includes the revision it was based on, and the server rejects with a conflict if it no longer matches. This is not full real-time collaborative merging, just enough to stop one save from silently overwriting another.
- Publish (POST /api/workflows/:id/publish) is a separate, explicit, non-debounced action that overwrites the single published copy with the current draft's contents. There's no version history to maintain: a run already in progress is unaffected because it captured its own steps_snapshot (C.3) when it started, not a live pointer to the workflow — so overwriting the published copy is safe by construction, not because anything was versioned.

C.6.5 State Management & Component Structure

- Canvas-local, ephemeral UI state (current selection, which panel is open, drag-in-progress position) lives in a lightweight store such as Zustand — it never needs to be persisted or synced.
- Server state — the draft config, the fetched step-type client, run history — is owned by React Query, so loading, autosaving and cache invalidation reduce to its normal mutation/query lifecycle rather than hand-rolled sync logic.

- All data reaches the frontend through plain REST calls, never server-pushed events. Step types, workflow draft, and run history are fetched (and, where something needs to feel live, polled on an interval or refetched on focus) — there are no WebSockets, no server-sent events, and no client-side event subscriptions anywhere in the frontend. One uniform request/response path is easier to reason about and debug than a REST path plus a separate event-driven one.
- Component structure stays small and generic throughout: WorkflowCanvas (React Flow shell) feeds StepNode (a single generic node type) feeds StepEditorPanel (a shared header/footer shell wrapping a generic SchemaForm or a registered custom editor) feeds Legend/Palette (a loop over the step client). None of these are written per step type.

C.6.6 Automatic Layout

Because a step's position is computed rather than stored, the canvas arranges itself: the author lays nothing out by hand, and the graph stays readable as a workflow grows. The layout runs over the real steps and edges on each render, and the "+" insert affordances and dashed "End" markers are placed relative to the step they belong to. Dragging is only for reordering — while a step is dragged it follows the cursor, and on release it either splices into the edge beneath it or settles back into place.

C.6.7 Undo & Redo

The draft is held as the current steps plus a history of previous states and a redo stack. Each edit records the previous state and clears the redo stack; undo and redo move between the two, and switching to another workflow clears the history so edits can't be undone across workflows. Since every edit is a plain transformation of `steps[]`, the history is just a stack of those snapshots. Undo and redo are available from the toolbar and from the usual keyboard shortcuts, and either one saves like any other edit.

C.6.8 Validation Before Publish

The builder checks a workflow's structure as it is edited and uses the result in three places: a marker on each step that has a problem, a panel listing every issue with a link to the step it concerns, and the publish action itself. Errors block publishing; warnings are shown but don't. The checks cover a step whose required configuration is missing or invalid (checked against the same schema the backend uses), a Goal that has an outgoing connection, a Go to action with no target, an edge pointing at a step that no longer exists, a step that can't be reached from the trigger, an unconnected branch, and a loop with no branch that can end it. Together they keep a structurally broken workflow from going live.

C.6.9 Managing Workflows

The builder handles the whole set of workflows: create a new one (which opens the trigger picker), rename it, delete it along with its runs, tasks and history, and move it between draft and published. After a create or delete, the next workflow to show is chosen directly rather than inferred from the refreshed list, so the selection never briefly lands on the wrong one.

C.6.10 Form Composition & the Field Design System

The step-editor layer has grown from bespoke, hand-rolled inputs into a small design system, composed the same way at every level — the same “declare once, render generically” idea as the step type registry (C.6.1),

applied one layer down to the fields inside each editor. (The shape mirrors what production automation builders do: their palette catalog is served as data and each action's form is declared as data.)

- Atomic field components — a handful of controlled design-system atoms (text, select, number, tag-list, checkbox, and a titled section container), each paired with its own data-logic hook (e.g. `useTextField`), render every input in the builder. The generic `SchemaForm` and the bespoke editors compose these same atoms, so there is no duplicated input markup and styling is consistent everywhere.
- A field registry — field kinds are registered in a small map (`kind → atom`), the design-system counterpart to the step type registry one level down: adding a new field kind is one atom + one hook + one registry line.
- Composition with a data boundary — a step editor is described declaratively as a set of field descriptors, optionally grouped into titled sections, and bound to the step's config by a `useStepForm` hook. Every change is funnelled through a single projection that persists only backend-meaningful keys: a field hidden by a conditional (e.g. an interview's room when the mode is "video") is dropped from what reaches the backend, and editor-only UI state (touched flags, in-progress buffers, fetched option lists) never crosses that boundary.
- Fractal structure, component + hook at every level — each atom is a component plus a hook; each step editor is likewise a thin, presentational component plus its own data-logic hook (e.g. `useScheduleInterview`, `useCommunicate`), so an editor's logic is isolated and testable in the same shape as the atoms it composes. Committed values stay in one `steps[]` tree, so undo/redo, autosave and conflict handling are unaffected.
- Descriptor composition — field descriptors are plain objects produced by small factories, so a shared group is reused by ordinary object composition, the idiomatic equivalent of a base-type/inheritance mechanism rather than a bespoke indirection.

C.7 Backend Architecture & Tech Stack

Also kept brief for the same reason as C.5 — a whole-system reference, not a backend design document.

Layer	Choice	Reasoning
Frontend	React + TypeScript, React Flow	Matches the required stack; React Flow fits the node/edge canvas already shown in the demo directly.
API	Node.js + TypeScript (NestJS or Express)	Keeps the stack in one language; NestJS's module structure maps cleanly onto Workflow / Run / Step services.
Database	PostgreSQL	Relational integrity between Workflow → Step → WorkflowRun → RunStepHistory; JSONB columns hold per-step-type config without a new table per type.
Scheduling / queue	Redis + BullMQ (or an equivalent durable job queue)	Delay steps need "run this in exactly 3 days, survive a restart" — a real job queue is the only reliable way to satisfy NFR-1.

Layer	Choice	Reasoning
File storage	S3 (or equivalent object storage), pre-signed upload URLs	Request Document needs somewhere to actually store uploaded files; pre-signed URLs let the browser upload directly without proxying files through the API.
Email	SES or SendGrid, behind a Communication Service	Abstracts the provider from the Step Executor, and is the natural place to add SMS/WhatsApp later (C.9).
Auth / roles	The school's existing SSO + a role table	Reuses whatever identity the admissions system already has for FR-12, rather than inventing a second one.

C.7.1 Implementation Notes

File uploads for Request Document are stubbed in the same way, and single sign-on and role enforcement (FR-12) are specified rather than built. Code quality is enforced with ESLint (flat config: typescript-eslint plus the React and hooks rule sets) and Prettier, and dynamic step config is typed as unknown and narrowed to a typed shape at each boundary rather than any. Alongside the step registry the backend serves the palette catalog (GET /api/step-categories); the Schedule Interview step is backed by an interviews table and an interview-status branch condition, with a demo-only endpoint to mark an interview complete.

C.8 Security, Privacy & Compliance

- All applicant/guardian PII flows through Communicate; uploaded files (Request Document) can be more sensitive still — ID proofs, transcripts, immunisation records — and need encryption at rest and their own retention rule rather than inheriting the audit log's default.
- Role-based access (FR-12) so only authorised staff can publish a live, applicant-facing workflow; view-only access for anyone who only needs to see status. The full audit trail (FR-10/NFR-4) doubles as the record needed if a family disputes ever having been reminded.
- the applicable regime depends on where the school operates

C.9 Additional Step Types & Features Worth Adding

Workflows 2 and 3 already use two proposed additions beyond the original seven step types — Request Document and Assign to Staff / Create Task (fully specified in C.4). The items below go further still — features the product would benefit from as schools push past these examples.

- Wait until condition/date — Delay today is duration-based. A "wait until this specific date" (e.g. a term start date not known when the flow is authored) would let flows key off calendar deadlines, not just relative waits.
- Webhook / integration step — to receive events directly from an external fee-payment provider, and to push data out to a school's student information system.
- Additional Communicate channels (SMS/WhatsApp) — many families are more responsive to text than email for time-sensitive reminders.

- Saved, reusable message templates — Communicate already offers recipient-scoped merge-field variables for one-off messages (C.4); a template library on top of that would let staff save and re-apply a whole worded message across workflows, instead of re-typing similar wording each time.
- Simulate/test mode — run a workflow against a fake or historical application before publishing, to see the emails and branch outcomes without waiting real days or emailing a real family.
- Per-workflow analytics — how many applications are at each step right now, how many hit each Goal, where they get stuck — turning run history (FR-10) into something staff can act on, not just inspect one applicant at a time.

C.10 Publish & Rollout

No version history to manage (see C.3, C.6.4) — a workflow is simply draft or published, and publishing overwrites the one published copy. The product-level policy on top of that mechanism:

- The structural checks in C.6.8 stop a broken workflow from being published at all, so the review that remains is about wording and intent rather than wiring.
- A small starter library of example flows (the demo already has "Load example") helps staff bootstrap new flows from the ones in Part B rather than a blank canvas.
- A lightweight second-person review before publish is worth considering, given these flows email applicants directly — a wrong condition or typo here is public-facing, not internal.

Appendix 1 — Step Type Legend

Step type	Description
Trigger	Starts the workflow.
Delay	Wait.
Branches	Split the path.
Communicate	Send an email.
Set status	Change the application's status.
Go to action	Jump to another step.
Goal	What counts as success.
Request Document *	Ask for and track an upload.
Assign / Create Task *	Assign the application to a person and track a task.
Schedule Interview *	Book an interview slot for the applicant and track it.

The three additions specified in this document (see C.4), alongside the original seven. The Scholarship Application Review workflow (B.4) uses nine together; the Applicant Interview Scheduling workflow (B.5) exercises the tenth, Schedule Interview.